Technical Overview — A.T.L.A.S v0.2.0

> **A.T.L.A.S** (Advanced Technology for Logical Autonomous Systems) is a continuously running personal AI agent built by XAGI Labs. This document covers how it is built — its architecture, tech stack, key components, and deployment.

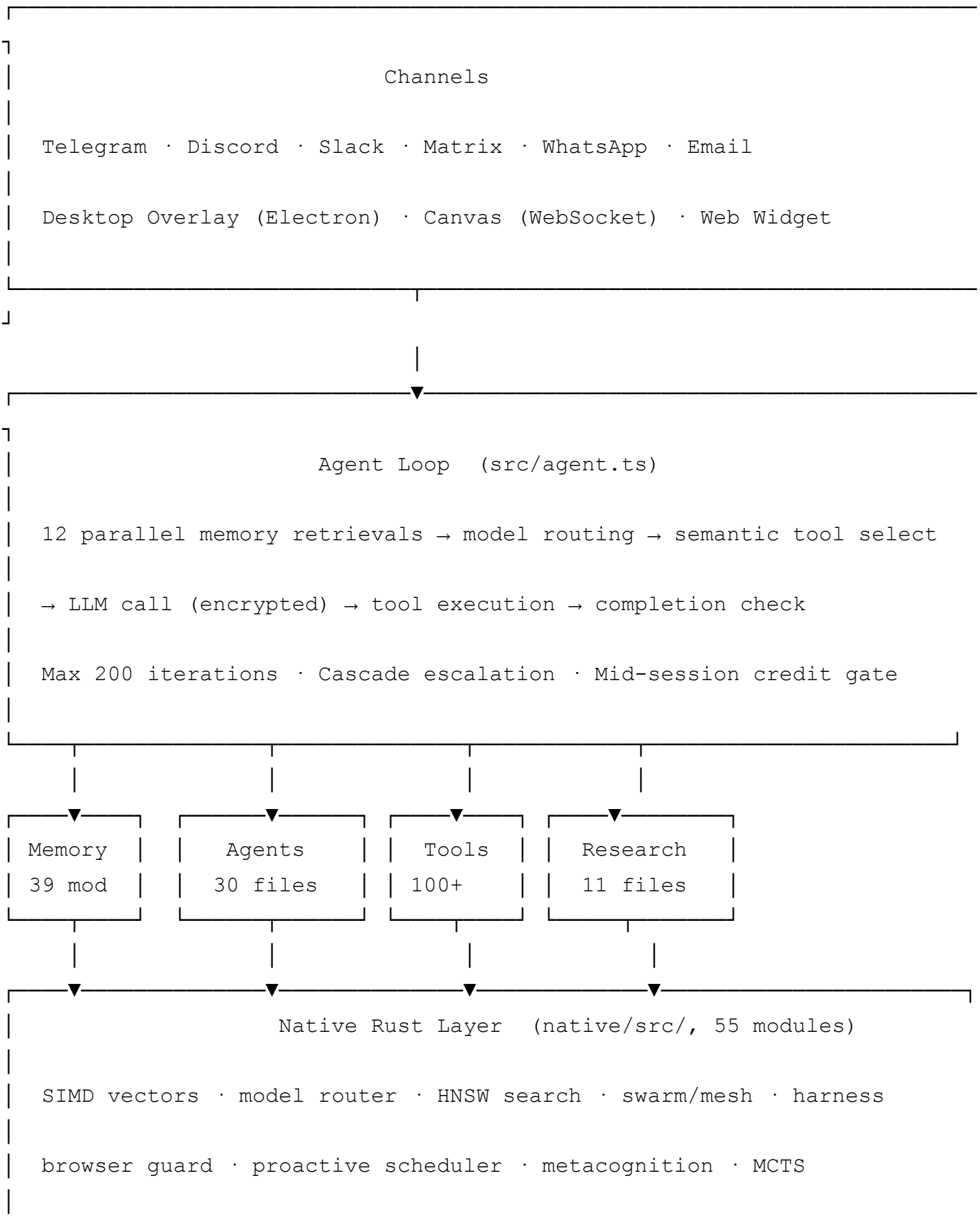
Table of Contents

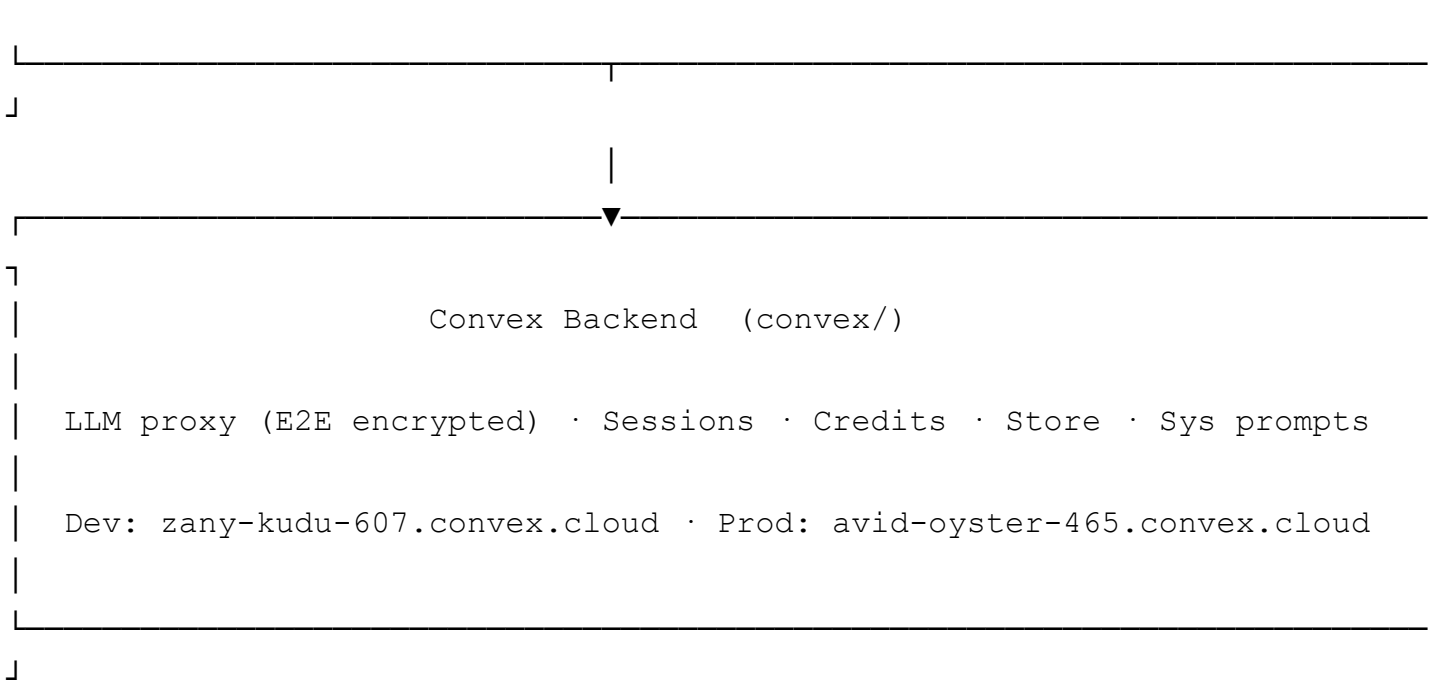
1. Architecture Overview
2. Tech Stack
3. Key Components & Services
    - 3.1 LLM Client & Encrypted Transport
    - 3.2 Model Router
    - 3.3 Tool Router
    - 3.4 Agent Loop
    - 3.5 Four-Tier Orchestration
    - 3.6 Memory System
    - 3.7 Native Rust Layer
    - 3.8 Convex Backend
    - 3.9 Browser Automation
    - 3.10 Coding Pipeline
    - 3.11 Deep Research Engine
    - 3.12 Proactive Engine
    - 3.13 Video Studio
    - 3.14 Meta-Harness Optimizer
    - 3.15 Voice Subsystem
    - 3.16 A2A Protocol
    - 3.17 Communication Channels
    - 3.18 Desktop App
    - 3.19 Marketing Brain
4. Data Flow
5. Security Architecture
6. Deployment
7. Configuration Reference

---

1. Architecture Overview

ATLAS is a **local-first** autonomous agent. The Node.js process runs on the user's machine, all memory stays on-device in SQLite, and the only outbound traffic is to a Convex backend that acts as an encrypted API-key proxy for LLM calls.





How it all connects

1. **Ingress** — A message arrives on any channel (Telegram, WhatsApp, Discord, Slack, Matrix, Email, Canvas, Overlay, Web Widget). The channel adapter extracts media (voice/image/video), runs sender authentication (user ID allowlist or persona check), and converts the inbound payload into a standardized `UserMessage` shape.
2. **Agent entrypoint** — The channel handler calls `runAgent(llm, message, onProgress, media)` in `src/agent.ts`. This function owns the entire conversation lifecycle; it is re-entrant (multiple channels may call it concurrently), but each invocation gets its own iteration counter and memory context.
3. **Memory fan-out (first iteration only)** — 12 memory retrieval systems fire in parallel via `Promise.all()`. Results are composed into an injection block under a ~2–3 K token budget and prepended to the system prompt. Each retrieval has a hard token ceiling so a single noisy source cannot starve the others.
4. **Model routing** — `chooseModel()` calls into native Rust (`native/src/model_router.rs`) with the current message, iteration count, error count, tool-call history, and media flags. The router returns a tier (Light / Pro / Heavy) and a `thinkingLevel` ("off" / "low" / "medium" / "high" / "max").
5. **Tool routing** — The query is embedded into a 384-dim MiniLM vector. Cosine similarity against the cached category embeddings selects the top-K tool categories. Only those tool schemas are loaded into the LLM context (typical: 15–36 tools out of 100+ available).
6. **LLM call** — The request is encrypted end-to-end with a per-session AES-256-GCM key derived via ECDH, posted to Convex, proxied to OpenRouter, and the response is decrypted client-side. API keys never appear on the client.
7. **Tool execution** — The agent parses `tool_calls` from the LLM response. Non-browser tools execute in parallel (`Promise.all`); browser tools execute sequentially because they share a

single Playwright `Page` object.

8. **Loop termination** — The loop continues until `finish_reason === "stop"`, the agent emits `TASK_COMPLETE`, the iteration cap (200) is hit, or the circuit breaker trips.
9. **Egress** — The final assistant message is streamed back to the originating channel using its own formatting adapter (Markdown → HTML for Telegram, blocks for Slack, embeds for Discord, plain for SMS-style channels).
10. **Post-session** — After the conversation settles (no follow-up within a threshold), `runReflection()` runs: extract coding strategies, detect goals, record outcomes into the self-model, and embed the turn into episodic memory.

Why local-first

ATLAS runs as a single long-lived Node.js process on the user's own machine. This choice is deliberate:

- **Privacy:** memory, embeddings, browser profile, and research artifacts never leave the device
- **Latency:** tool calls (shell, browser, file edits) run against local resources with zero network hop
- **Resilience:** ATLAS continues to function for memory recall, file edits, and local tools even when offline — only LLM calls require the Convex proxy
- **Cost:** expensive operations (vector search, reranking, tree-sitter parsing) run on local hardware instead of burning cloud compute

---

2. Tech Stack

Layer	Technology	Version	Notes
Runtime	Node.js	>= 20	ES Modules throughout (`"type":"module"`)
Language	TypeScript	^5.7	Strict mode, ES2022, NodeNext resolution
Native acceleration	Rust + napi-rs	edition 2021	Compiled to `.node` binary per platform
SIMD math	`wide` + `simsimd`	0.7 / 6.5	~15-20x speedup vs pure JS
Parallelism	`rayon`	1.10	CPU-parallel batch ops in Rust
Encryption (native)	`aes-gcm` + `scrypt`	0.10 / 0.11	AES-256-GCM + Argon2-class KDF
Local database	SQLite (better-sqlite3)	^12.6	WAL mode, 64 MB cache, 256 MB mmap

---

Layer	Technology	Version	Notes
Serverless backend	Convex	^1.34	Auth, LLM proxy, credits, store
LLM routing	OpenRouter	—	All providers via one proxy
Embeddings	`@xenova/transformers`	^2.17	MiniLM L6 v2 (384-dim), Nomic optional
Browser automation	Playwright	^1.50	Chromium headless, persistent profile
Chrome bridge	Custom extension	MV3	Real Chrome via WebSocket IPC
Desktop app	Electron	^40+	macOS + Windows, auto-updater
Telegram	grammY	^1.35	Bot API
Discord	discord.js + voice	^14.26 / ^0.19	Text + voice
Slack	@slack/bolt	^4.7	Socket Mode
WhatsApp	@whiskeysockets/baileys	^7.0-rc.9	WhatsApp Web protocol
Email	IMAPFlow + Nodemailer	^1.2 / ^8.0	IMAP read + SMTP send
Video processing	fluent-ffmpeg	^2.1	FFmpeg wrapper
Code parsing	tree-sitter	^0.22	TS, JS, Python, Rust, Go
Schema validation	Zod	^4.3	All tool parameter schemas
Linter/formatter	Biome	^2.4	Replaces ESLint + Prettier
Test runner	Vitest	^3.2	Unit + integration tests
Swift helpers	Swift 5.9	macOS 13+	OCR (`ocr_screen.swift`), GPS (`get_location.swift`)
MCP	@modelcontextprotocol/sdk	^1.9	External tool providers

LLM Providers (via OpenRouter)

Provider	Models used
Anthropic	Claude Opus 4.6, Sonnet 4.6, Sonnet 4.5, Haiku 4.5
Google	Gemini 3.1 Pro, 2.5 Flash, 3 Flash Preview, 3.1 Flash Lite, 2.0 Flash, 2.5 Flash Lite

---

Provider	Models used
Amazon	Nova Pro v1, Nova Lite v1, Nova Micro v1
OpenAI	GPT-5 Image (image generation only)

All LLM API keys live exclusively in Convex environment variables.

---

3. Key Components & Services

3.1 LLM Client & Encrypted Transport

Files: `src/llm.ts` , `src/crypto-transport.ts`

Every LLM call goes through a two-layer security stack:

Layer 1 — ECDH session key exchange
When the desktop app starts (or when a new session begins), the client generates an ephemeral P-256 key pair and sends the public key to Convex via `/api/auth/session` . Convex returns its own ephemeral public key. Both sides derive the same 256-bit shared secret using ECDH + SHA-256. This shared key is stored with the session record.

Layer 2 — AES-256-GCM payload encryption
Every subsequent request to `/api/llm/chat` is encrypted with the derived key. The envelope format is `{ iv, ct, tag, ts, rid }` (all base64). The server decrypts, validates the timestamp (90-second replay window), forwards to OpenRouter with its own API key, encrypts the response, and returns it. API keys never travel to or from the client.

Semantic response cache (`src/llm-cache.ts`): Simple queries are cached in a 200-entry LRU using MiniLM cosine similarity. A hit requires similarity > 0.88 and age < 3 minutes. Tool-call requests and extended-thinking requests are never cached. The cache key is the query's 384-dim embedding plus a hash of the skill slugs and model hint; mismatched skills or different models bypass the cache.

Adaptive language matching (`src/language.ts`): Every request is analyzed for writing system (Devanagari, CJK, Cyrillic, Arabic, Hangul, Thai, and 10+ Latin-script languages + Hinglish romanization). A `Respond in {language}` directive is injected automatically. Detection uses Unicode script analysis as the primary signal, with keyword heuristics as a tiebreaker (e.g. distinguishing Hindi-in-Latin-script from English).

Thinking levels — the `thinkingLevel` parameter controls extended reasoning budgets on supported providers:

Level	Budget	When chosen
`off`	0	Casual chat, simple classification
`low`	1 K tokens	Standard tool loops
`medium`	4 K tokens	Complex reasoning, planning
`high`	12 K tokens	Heavy research synthesis, design docs
`max`	32 K tokens	Deep research final reports, complex coding tasks

Thinking level is inferred from task type by the Rust router; the LLM client never overrides it unless `ATLAS_NO_CODING_THINKING=true` for coding tasks.

Skills injection — when the caller passes `skillSlugs`, the Convex server prepends the matching markdown skill content to the system prompt (from the `system_skills` table). This lets specialized domains (e.g. Convex development, video editing) get additional context without the client ever holding the skill content.

Response format — callers can pass `responseFormat: "json_object"` to force valid JSON output. The Convex proxy translates this to `response_format: { type: "json_object" }` for OpenAI-compatible providers or `responseMimeType: "application/json"` for Gemini routes. The `extractJSON()` utility in `src/utils/json-extract.ts` provides a fallback for providers that don't support the flag — it strips markdown fences and recovers JSON from mixed output.

3.2 Model Router

Files: `src/model-router.ts`, `native/src/model_router.rs`

The router is implemented in Rust and exposed to TypeScript via napi-rs. It selects a model and thinking budget based on:

- Current iteration count and error count
- Task type (`coding`, `research`, `tool_loop`, `error_recovery`, etc.)
- Whether the query involves vision, audio, or media
- User hint (`"simple"` / `"complex"` strings in the message)
- Conversation mood (`casual`, `focused`, `emotional`, `urgent`)

Cost tiers:

Tier	Credits	Models	When used
Light	0.25	Haiku 4.5, Gemini Flash, Nova Micro/Lite, Gemini Flash Lite	Casual chat, simple retrieval, tool routing

Tier	Credits	Models	When used
Mid	0.50	Nova Pro, Gemini 2.5 Flash	Moderate tool use, summaries
Pro	1.0	Sonnet 4.6, Sonnet 4.5, Gemini 3.1 Pro	Browser automation, coding, multi-tool
Heavy	3.0	Opus 4.6	Complex reasoning, deep research, error recovery

Cascade escalation: If errors accumulate or the circuit breaker trips (5 consecutive failures or 3 min without progress), the router automatically upgrades to the next tier. This works in sub-agents too, not just the main loop.

Coding escalation (`maybeEscalateForCoding`): Any non-trivial coding intent forces tier ≤ Pro. Extended thinking budget is injected unless `ATLAS_NO_CODING_THINKING=true` .

Complete model catalog (`MODELS` constant in `src/llm.ts`):

Symbol	Model ID	Display name	Tier
`PRO`	`anthropic/claude-opus-4.6`	Opus 4.6	Heavy
`SONNET`	`anthropic/claude-sonnet-4.6`	Sonnet 4.6	Pro
`SONNET_45`	`anthropic/claude-sonnet-4.5`	Sonnet 4.5	Pro
`HAIKU`	`anthropic/claude-haiku-4.5`	Haiku 4.5	Light
`GEMINI_PRO`	`google/gemini-3.1-pro-preview-customtools`	Gemini 3.1 Pro	Pro
`GEMINI_25_FLASH`	`google/gemini-2.5-flash`	Gemini 2.5 Flash	Mid
`FLASH`	`google/gemini-3-flash-preview`	Gemini 3 Flash	Light
`FLASH_LITE`	`google/gemini-3.1-flash-lite-preview`	Gemini 3.1 Flash Lite	Light
`GEMINI_25_FLASH_LITE`	`google/gemini-2.5-flash-lite`	Gemini 2.5 Flash Lite	Light
`GEMINI_20_FLASH`	`google/gemini-2.0-flash-001`	Gemini 2.0 Flash	Light
`NOVA_PRO`	`amazon/nova-pro-v1`	Nova Pro	Mid
`NOVA_LITE`	`amazon/nova-lite-v1`	Nova Lite	Light
`NOVA_MICRO`	`amazon/nova-micro-v1`	Nova Micro	Light

Fallback chains (`MODEL_FALLBACKS`): when a primary model returns 5xx, rate-limits, or times out, the router tries the next model in its fallback list. Each model has a 2-model fallback chain ordered by similarity of capability.

Horizontal failover — the Rust router returns a `horizontalFailover` list derived dynamically from model metadata (tier, speed, context window, tool support). The TS client iterates through this list on error before escalating to a higher tier.

3.3 Tool Router

Files: `src/tool-router.ts` , `native/src/tool_router.rs`

Tool selection uses semantic similarity — not keyword matching.

At boot: Each of 20+ tool category descriptions is embedded with MiniLM (384-dim vectors) and cached in the Rust module.

Per request: The user query is embedded → cosine similarity computed against all category embeddings → top-K categories selected → tool schemas for those categories loaded into the LLM context.

A `request_tools` meta-tool lets the agent ask for additional tool categories mid-task without restarting. Co-occurrence learning tracks which tools are used together and boosts correlated categories.

Categories: core, memory, browser, web_search, file_system, shell, os_control, system_controls, media, communication, scheduling, app_management, research, coding, long_tasks, google_api, social_media, voice, document, integrations.

FunctionGemma fallback (`src/models/function-gemma.ts`): An optional 270M local model via Ollama (85% accuracy) can be used for zero-cost tool routing when the LLM call budget is tight.

Analytics (`src/tool-router-analytics.ts`): Tracks tool usage patterns, selection accuracy, and category hit rates for the Meta-Harness optimizer.

3.4 Agent Loop

File: `src/agent.ts`

The core execution engine. Each call to `runAgent()` runs through this pipeline:

1. **Account gate** — check credits via Convex; block if exhausted
2. **Message preprocessing** — strip internal tags, prune context window
3. **First iteration only:** 12 memory retrieval systems fire in parallel:
    - Identity profile, behavioral traits, session summary
    - Self-state, emotional context, episodic milestones
    - Relationship tone, anticipation signals, core memory

    - Query-specific recall (HNSW vector search)
    - Task learnings, knowledge graph entities
    - Each has a token budget; total injection ~2-3K tokens
4. **Each iteration:** `chooseModel()` → semantic tool select → LLM call → tool execution → check for `finish_reason: stop` or `TASK_COMPLETE` signal
5. **Tool execution:** Browser tools run sequentially (shared page state); all other tools run in parallel
6. **Cascade escalation:** Errors accumulate → auto-upgrade model tier
7. **Max 200 iterations;** mid-session credit check every 5 tool calls

Circuit breaker (`src/agents/circuit-breaker.ts`): Trips on 5 consecutive failures, 8 no-progress iterations, or 3-minute stall. States: CLOSED → OPEN → HALF-OPEN with escalating cooldowns (30s → 60s).

Metacognition (`src/agents/metacognition.ts`): The loop tracks signals every iteration — token usage, tool error rate, progress delta, confidence — and asks "should I stop, escalate, ask for help, or retry?" at each decision point. When it detects pathological patterns (loop detection on tool call sequences, monotonic confidence decay, plateau on progress), it triggers intervention: either switch role, request human input via `human-interrupt.ts` , or gracefully exit with a partial result.

Plan mode (`src/agents/plan-mode.ts`): A runtime-gated state machine with four states:

State	Writes allowed?	How to enter	How to exit
`act`	Yes	Default	—
`plan`	No (read-only tools)	`enter_plan_mode({ reason })`	`exit_plan_mode({ plan })`
`spec`	No	`enter_spec_mode({ title })`	`finish_spec({ spec })`
`awaiting_approval`	Design-docs dir only	Auto, on `generate_design_docs` success	`approve_design_docs({ user_quote })`

Each state has an explicit `*_ALLOWED_TOOLS` list in `plan-mode.ts` ; `filterToolsFor*` strips tools before the LLM call, and `checkWritePathInApprovalMode()` enforces at execution time.

Context pruning (`src/memory/context-pruning.ts`): When total message history tokens exceed the configured cap (`ATLAS_CONTEXT_CAP` , default 8000), older messages are summarized rather than dropped. The pruner preserves:

- All system reminders
- The most recent user message
- The last N tool calls + results (N dynamic based on iteration count)

- A compressed rolling summary of earlier turns

System reminders (`src/agent/system-reminders.ts`): Injected into the message stream (not the system prompt) at specific decision points to nudge behavior without bloating context. Examples: "You are at iteration 150/200 — start wrapping up"; "Plan mode active — write tools blocked"; "Coding complexity: complex — follow the 4-phase flow".

3.5 Four-Tier Agent Orchestration

Directory: `src/agents/` (30 files)

Tier	Entry point	Max scale	Purpose
Sub-Agent	`sub-agent.ts`	—	Specialized role (researcher/coder/browser/system/general), scoped tool subset, loop detection
Swarm	`swarm.ts`	20 steps	Decomposes task → executes via Mesh → evaluator-optimizer loop with up to 3 replans
Mesh	`mesh.ts`	Unlimited steps	Parallel DAG: independent steps run concurrently, dependent steps sequentially; critical path scheduling
Long-Task	`long-task.ts`	100 phases × 200 iter × 30 min	Multi-phase executor; checkpoints every 60s to `data/long-tasks/{id}.json` ; resumable after crash

Reliability layers:

- `error-recovery.ts` — retry → rephrase → reroute (switch role) → skip
- `evaluator.ts` — quality scoring (completeness/accuracy/coherence/actionability 0-100); retry if < 60
- `replanner.ts` — revises remaining plan when failures or new info detected
- `metacognition.ts` — SOFAI-LM-style self-monitoring; detects loops, low confidence, and triggers interventions
- `multi-critic.ts` — multi-perspective debate on solution quality
- `code-critic.ts` — compile + lint + test + SAST (Semgrep) in parallel; weighted consensus score
- `agent-worktree.ts` — git worktree isolation for parallel coder steps (opt-in via `ATLAS_AGENT_WORKTREE=true`)
- `mixture.ts` — Mixture-of-Agents: 3 parallel proposals blended for complex tasks
- `speculative.ts` — races primary + alternative approach; first success wins
- `a2a.ts` — Google A2A protocol; exposes ATLAS as a callable agent to external systems

Shared context: `blackboard.ts` provides a per-swarm key-value store for inter-agent data passing.

Human-in-the-loop: `human-interrupt.ts` pauses dangerous operations (delete, deploy, send email) and waits for user confirmation (default 5-minute timeout with configurable default action).

3.6 Memory System (39 modules)

Directory: `src/memory/`

All memory is local SQLite (`data/memory.db`) with WAL journal mode, 64 MB page cache, and 256 MB mmap. No memory data leaves the user's machine.

Storage layers:

Module	Purpose	Key detail
`sqlite.ts`	Primary store	`remember()` , `recall()` , `getCoreMemory()`
`vector-index.ts`	HNSW ANN search	M=16, EF=200/50, Float16 quantization
`knowledge-graph.ts`	Entity-relationship graph	Bi-temporal (valid_at/invalid_at/expired_at)
`embeddings.ts`	Vector generation	MiniLM 384-dim (default), Nomic 768→256 optional
`emotional.ts`	Emotional trajectory	Exponential decay (λ=0.05), 30-day max
`episodic.ts`	Milestone events	achievement, struggle, decision, milestone
`traits.ts`	Behavioral traits	Confidence-weighted, 600-token budget, conflict detection
`identity.ts`	User profile	JSON Resume format; persisted to `user_profiles`
`working-memory.ts`	Short-term buffer	20 items, 3000-token budget, attention-scored
`hopfield-memory.ts`	Associative recall	Continuous Hopfield network, similarity > 0.3 gate
`world-model.ts`	World state	Per-entity accessibility scores, EMA response times
`user-model.ts`	User model	Aggregates identity + traits + KG into unified prediction
`active-inference.ts`	Predictive inference	Free-energy minimization; explore vs exploit (Nature Comms Dec 2025)

Module	Purpose	Key detail
`reflection.ts`	Nightly consolidation	SAC-DR; trait extraction, HMAC audit log
`self-evolving.ts`	Synaptic pruning	Strength-based: weak→delete, medium→weaken, strong→strengthen
`task-learnings.ts`	Task strategies	Per-(category, pattern, context) learning; merges on re-encounter
`codebase-index.ts`	Repo sidecar	Per-repo SQLite at `data/codebase-index/{hash}.db` ; tree-sitter parse

Retrieval fusion: Results from multiple sources are blended using RRF (Reciprocal Rank Fusion) + recency exponential decay + importance weights + SAN/Hopfield boost multipliers. A cross-encoder reranker makes the final pass.

Memory decay: Runs every 6 hours. Self-evolving synaptic pruning uses a strength score: `importance × log₂(1 + access_count) × exp(-0.01 × days_since_access)` . Memories below 30% of median strength are deleted.

Dream consolidation (`reflection.ts` , `dreaming.ts`): When the user has been idle for 2+ hours during 11 pm–6 am, ATLAS enters dream mode — consolidates memories, runs pending tasks via sub-agents, proposes self-edits for repeated failures, and queues results for morning delivery.

Memory categories — every entry stored via `remember()` is tagged with a category for retrieval scoping and decay tuning:

Category	Purpose	Decay
`identity`	User profile (name, role, preferences)	None (permanent)
`preference`	Things the user likes / dislikes	Slow (90 d)
`fact`	Objective fact about the user's world	Slow (180 d)
`episode`	Conversation milestone	Medium (30 d)
`skill`	Something the user is learning / good at	Slow (90 d)
`relationship`	Person, colleague, family	Slow (90 d)
`code` / `code_*`	Coding strategy (preserved from pruning)	None
`task`	Pending / completed task	Short (7 d)
`observation`	Ambient passive observation	Short (3 d)

Bi-temporal knowledge graph — `knowledge-graph.ts` stores every fact with three timestamps: `valid_at` (when the fact started being true), `invalid_at` (when it stopped), and `expired_at`

(when ATLAS last confirmed the fact). This allows queries like "what did the user believe 3 days ago?" to return historically accurate answers.

Vector index internals — `vector-index.ts` uses HNSW with:

- `M = 16` (max connections per layer)
- `efConstruction = 200` (beam width during insertion)
- `efSearch = 50` (beam width during query)
- Float16 quantization of embeddings to halve memory footprint (2× more vectors in the same RAM)
- Incremental batch insertion every 50 embeddings or 5 seconds

Retrieval pipeline — when `recall()` is called:

1. **Candidate generation** — three parallel passes: HNSW vector search (top 50), BM25 full-text search (top 30), recency scan (top 20 most recent within the requested scope)
2. **RRF fusion** — combine the three rankings with `score = Σ 1/(k + rank_i)` , k=60
3. **Feature expansion** — add importance, access count, recency decay, SAN activation boost, Hopfield associative boost
4. **Cross-encoder rerank** — a small cross-encoder (`Xenova/ms-marco-MiniLM-L-6-v2`) re-ranks the top 30 by semantic relevance to the query; blended with the RRF score via `crossEncoderBlend` harness dimension (default 0.6)
5. **Diversity filter** — MMR-style deduplication to avoid returning near-identical memories
6. **Token-aware trimming** — pack results to fit the caller's token budget, preferring shorter high-score entries

---

3.7 Native Rust Layer (55 modules)

Directory: `native/src/`

Built with napi-rs, compiled to platform-specific `.node` binaries. TypeScript loads the binary at runtime; all subsystems fall back to pure-JS implementations if the binary is unavailable.

Build profile: `opt-level=3` , LTO enabled, `codegen-units=1` , `panic=abort` , binaries stripped.

Module groups:

Group	Modules	What it accelerates
Vector math	`vector.rs` , `batch.rs` , `hnsw.rs`	SIMD dot-product, cosine similarity, ANN search

Group	Modules	What it accelerates
Routing	`model_router.rs`, `tool_router.rs`, `tool_analytics.rs`, `query_classifier.rs`	Model/tool selection scoring
Memory	`memory_state.rs`, `memory_tiers.rs`, `knowledge_graph.rs`, `hnsw.rs`, `hopfield.rs`, `spreading.rs`, `temporal_cache.rs`, `consolidation.rs`, `sleep_gate.rs`, `surprise_gate.rs`, `reflection_core.rs`, `session_memory.rs`, `working_memory.rs`, `task_learnings.rs`, `traits_engine.rs`	Hot-path memory ops
Agents	`agent_core.rs`, `swarm.rs`, `mesh.rs`, `sub_agent.rs`, `long_task.rs`	Orchestration state tracking
Reasoning	`metacognition.rs`, `mcts_engine.rs`, `active_inference.rs`, `logic_engine.rs`, `process_reward.rs`, `pattern_speculator.rs`, `prompt_evaluator.rs`	Inference and planning
World/user models	`world_model.rs`, `user_model.rs`, `credibility_model.rs`	Predictive state
Proactive	`proactive_scheduler.rs`, `proactive_monitor.rs`, `proactive_intent.rs`, `inner_thoughts.rs`, `goal_engine.rs`, `habit_detector.rs`, `notification_tracker.rs`	Intent queue, rate limiting, DND
Browser	`browser_agent.rs`, `browser_guard.rs`, `chrome_bridge.rs`, `ip_guard.rs`	Element scoring, stealth, CAPTCHA detection
Research	`research_engine.rs`, `info_gain.rs`, `citation_graph.rs`, `skill_scorer.rs`	Research acceleration
Harness	`harness_optimizer.rs`	NSGA-II Pareto, GP surrogate, TextGrad scoring
Security	`security.rs`	Encryption helpers

Performance numbers (from `native/README.md`):

Operation	JS	Native	Speedup

Operation	JS	Native	Speedup
cosine similarity (768-dim)	12 µs	0.8 µs	15×
findTopK (1000 vectors)	15 ms	0.9 ms	17×
batch similarity	14 ms	0.7 ms	20×

3.8 Convex Backend

Directory: `convex/`

Convex serves as the secure cloud layer. The Node.js client connects to it via the Convex SDK; API keys for LLMs live only in Convex environment variables and are never sent to the client.

Database tables (11 total, defined in `convex/schema.ts`):

Table	Purpose
`accounts`	OTPless identity, plan (free/pro/enterprise), AI credits, activations
`sessions`	24h TTL tokens; ECDH shared key for E2E encryption; device fingerprint
`rate_limits`	Sliding window — 60 requests per 60 seconds per session
`developers`	Store publisher accounts (email + bcrypt, approval workflow)
`admins`	Admin panel users with role + permissions
`store_items`	Published skills, tools, MCP servers (slug, type, category, tags, pricing)
`submissions`	Pending store review queue
`store_installs`	User install records with auto-update preference
`store_reviews`	1–5 star ratings with helpful votes
`store_activity_log`	Analytics for admin dashboard
`system_prompts`	Server-side identity/rules/coding/browsing prompts (versioned)
`system_skills`	Built-in skill definitions served at runtime

HTTP endpoints (`convex/http.ts`):

Endpoint	Method	Purpose
`/api/llm/chat`	POST	Authenticated LLM proxy with Anthropic prompt caching

---

Endpoint	Method	Purpose
`/api/auth/session`	POST	OTPless → session token + optional ECDH key exchange
`/api/account/status`	GET	Credit balance and account status
`/api/auth/refresh`	POST	Extend session TTL
`/api/auth/revoke`	POST	Logout
`/api/system/prompt`	GET	Fetch system prompt by slug
`/api/search/tavily`	POST	Tavily search proxy
`/api/search/exa`	POST	Exa search proxy
`/api/keys/voice`	GET	Google/OpenAI/Deepgram keys for voice
`/api/spotify/app-token`	POST	Spotify client credentials
`/api/app/runtime-key`	POST	XOR key delivery for desktop app protection
`/api/updates/latest-mac.yml`	GET	Electron auto-updater metadata
`/api/config/secrets`	GET	HuggingFace token, CAPTCHA keys

Prompt caching: The `/api/llm/chat` handler adds `cache_control` breakpoints to Anthropic model requests (system prompt, tools schema, conversation history), enabling up to 90% cost reduction on repeated context via Anthropic's prompt caching feature.

Credit costs (per LLM call):

Tier	Cost	Examples
Light	0.25 cr	Haiku 4.5, Gemini Flash, Nova Micro/Lite
Mid	0.50 cr	Nova Pro, Gemini 2.5 Flash
Pro	1.00 cr	Sonnet 4.6, Gemini 3.1 Pro
Heavy	3.00 cr	Opus 4.6, GPT-5 Image

New accounts start with 100 free credits.

Atomic credit deduction — OCC (Optimistic Concurrency Control) is used to prevent race conditions:

```
// convex/accounts.ts (simplified)
await ctx.db.patch(accountId, {
  ai_credits: account.ai_credits - cost,
});
```

Convex's OCC serializes concurrent mutations on the same document, so two LLM calls arriving simultaneously cannot both spend the last credit. If a mutation conflicts, Convex retries automatically.

Session token lifecycle:

1. Client calls `POST /api/auth/session` with OTPless user ID (and optional ECDH public key)
2. Server generates a random 256-bit token, computes the shared AES-256 key (if client sent pubKey), and inserts a row into `sessions` with `expiresAt = now + 24h`
3. Client stores token in `safeStorage` (Electron) or memory (CLI)
4. Each subsequent request sends `Authorization: Bearer <token>`
5. Server validates: token exists → not expired → matches device fingerprint (if set) → not revoked
6. Expired sessions are garbage-collected by a Convex cron job every hour

System prompt management — prompts are source-of-truth in `prompts/*.md` in the repo, seeded to the `system_prompts` table via `scripts/seed-system-content.ts` (needs `ADMIN_API_KEY`). The client pulls them on demand:

```
// src/config/system-prompts.ts
const prompt = await fetchSystemPrompt('coding');  // HTTP GET /api/syste
```

In-flight request coalescing ensures concurrent callers requesting the same slug share a single HTTP round-trip. A single-flight auth refresh handler prevents auth-token-expired requests from stampeding the refresh endpoint.

Seeded slugs: `identity`, `rules`, `rules-mac`, `rules-windows`, `browsing`, `thinking`, `coding` (Phase 4 tool-use workflow), `spec` (spec-mode guidance).

Store marketplace flow:

1. Developer registers via portal → `developers` row with `status: "pending"`
2. Admin approves → status flips to `"approved"`; developer can now submit items
3. Developer submits skill/tool/MCP server → `submissions` row + file uploaded to Convex storage
4. Admin reviews → approve: row moves to `store_items` and becomes publicly discoverable; reject: submission stays in queue with notes
5. User installs → `store_installs` row created; hot-reload injects the item into the running agent
6. User reviews → `store_reviews` row; affects item ranking and featured placement

3.9 Browser Automation

Directory: `src/tools/browser/` (20 files)

ATLAS supports two browser backends behind a unified `BrowserBackend` interface:

Playwright backend (`playwright-backend.ts`): Headless Chromium with a persistent profile at `data/browser-profile/` . Default for most use cases.

Chrome extension backend (`chrome-backend.ts` , `chrome-bridge.ts`): A WebSocket bridge to the user's real Chrome browser via a companion extension (Manifest V3). The `ChromePageProxy` class mimics Playwright's Page API, so the rest of the code doesn't know which backend is active.

Stealth (`browser_guard.rs`): The Rust module compiles a 13-patch stealth initialization script that:

- Sets `navigator.webdriver = false`
- Injects a realistic `window.chrome` stub (runtime, loadTimes, csi, app)
- Fills `navigator.plugins` with realistic entries
- Removes `__playwright` , `__puppeteer` , `$cdc_*` markers
- Cleans CDP stack traces from `Error.stack`
- Fixes headless window dimension mismatch
- Simulates human hardware (`hardwareConcurrency=8` , `deviceMemory=8`)
- Generates session-stable, platform-appropriate Chrome user agents

Because the script is compiled into the Rust binary, it cannot be read from unpacked TypeScript.

Human timing simulation: `humanTypingDelayMs()` uses WPM-based per-character delays with ±30% jitter and extra pauses for punctuation. `humanClickDwellMs()` simulates 150–400 ms button holds. Mouse movement follows Fitts's law via `humanMouseParams()` .

CAPTCHA solving (`captcha.ts`) — 4-tier waterfall:

Tier	Method	Cost	Notes
1	Turnstile passive stealth	Free	Waits up to 20s for frame to hide
2	reCAPTCHA v2 audio bypass	Free	Downloads MP3, transcribes via Whisper/Gemini
3	NopeCHA API	Free (100/day)	`NOPECHA_API_KEY` env var
4	2captcha	Paid	`CAPTCHA_API_KEY` env var

Video control (`video-control.ts`): Controls YouTube and HTML5 video players with 3-tier fallback: JS Player API → keyboard shortcuts → hover+click UI. A 10-second native cache (LRU 16 entries) avoids redundant state queries.

OpenCLI tools (`opencli-tools.ts`): `browser_batch_fetch` fetches N URLs in a single `page.evaluate()` call with configurable concurrency (default 5, max 20). `browser_auto_scroll` uses MutationObserver for intelligent scroll-and-settle detection.

Element compression (`browser_guard.rs`): Two-tier bounding-box dedup + spatial dedup (30 px radius) + consecutive group folding reduces DOM snapshots by ~15% before sending to the LLM, using the OpenCLI `[N]<tag>text` format.

Mode-based tool switching (`mode-tools.ts`):

Mode	What the agent sees	When used
`fast`	Text-only DOM snapshot (compressed, IDs visible)	Default — cheaper, faster
`vision`	Screenshot + OCR overlay + SoM (Set of Marks) annotations	Complex pages, visual-only elements

The agent calls `set_browser_mode({ mode: "vision" })` when text-only DOM inspection fails to find the target element; falls back to `fast` mode when the task is done.

Browser memory (`src/tools/browser-memory.ts`): Persists site-specific learnings across sessions — element selectors that worked, cookie banners that were dismissed, anti-bot gates that were bypassed. Stored in the main `memories.db` with category `browser_strategy` .

Strategy save/recall (`strategy-save.ts` , `strategy-recall.ts`): When the agent finds a good approach for a site ("on Reddit, click 'Show all comments' before scraping"), it saves that strategy. On the next visit to the same domain, the strategy is pre-injected into context.

Profile sync (`profile-sync.ts`): Synchronizes the persistent Playwright profile across devices by exporting cookies, localStorage, and IndexedDB to a portable archive.

3.10 Coding Pipeline

Directory: `src/tools/coding/` , `src/agents/plan-mode.ts`

Complexity classification

Every coding request is classified on the first iteration:

Level	Criteria	Behavior
Trivial	Add log, rename, typo fix, bump version	No design docs, stays on Light tier

---

Level	Criteria	Behavior
Standard	Direct implementation	Pro tier, straight to code
Complex	Refactor, new system, build verb + high-stakes domain, > 400 chars	Full 4-phase flow below

Complex task flow

1. **Design docs** — `generate_design_docs` writes PRD.md + TECHNICAL.md + ARCHITECTURE.md under `A.T.L.A.S Files/Documents/design-docs/<slug>/`. Auto-transitions to `awaiting_approval` state.
2. **Approval gate** — Write tools are blocked except in the design-docs directory. The agent calls `approve_design_docs({ user_quote })` when the user approves.
3. **Implement** — Standard coder loop on Pro tier. Git worktree isolation if `ATLAS_AGENT_WORKTREE=true` and ≥ 2 parallel coder steps.
4. **Walkthrough** — `demo_project` auto-detects stack, launches runtime, records screen, generates WALKTHROUGH.md via `git log` + `git diff`.

Supporting tools

- **LSP** (`lsp.ts`): `goToDefinition`, `findReferences`, `hover`, `documentSymbol`; auto-attached to edit results as `post_edit_diagnostics` unless `ATLAS_NO_POST_EDIT_DIAGNOSTICS=true`
- **Codebase index** (`codebase-index.ts`): Per-repo SQLite sidecar at `data/codebase-index/{hash}.db`; tree-sitter parses TS/JS/Python/Rust/Go; BM25 FTS5 over symbol names; depth auto-picks: ≤500 files → call graph, ≤5000 → imports, else → symbols
- **git_op** (`git-op.ts`): 4-tier safety model — safe (status, commit, branch) / confirm (merge, cherry-pick) / destructive (reset, push) / blocked; protected branches `main`, `master`, `production`, `release/*`
- **Code critic** (`code-critic.ts`): compile (weight 0.4) + test (0.4) + lint (0.2) + SAST/Semgrep (0.15) + UI verify; fires in swarm synthesis for coder steps; 90s wall-clock budget

---

3.11 Deep Research Engine

Directory: `src/research/` (11 files)

A 3-phase background research loop that runs as a separate async task.

Phase 1 — Discovery (40% of source budget): Broad multi-engine search. Generates competing hypotheses (`hypothesis-engine.ts`) with Bayesian confidence updates. Each hypothesis has

designed `searchFor` and `searchAgainst` queries.

Phase 2 — Deep reading (40%): In-depth page analysis. Fetches full content, extracts findings, follows `followUpQueries`.

Phase 3 — Verification (20%): Cross-references findings. `crossReferenceFindings()` assigns confidence: strong (≥2 sources), moderate (1), weak (0).

Search engines (12 total): Google, Brave, DuckDuckGo, Bing, SearXNG, Mojeek, Tavily, Exa, Wikipedia, OpenAlex, Semantic Scholar, arXiv.

Engine health: Exponential backoff on failures: 15s → 30s → 60s → 120s → 240s (5 min cap). Failed engines are bypassed automatically.

Source ranking (`scoring.ts`): Weighted score = domain quality (0.25) + finding density (0.22) + content depth (0.18) + credibility (0.12) + recency (0.13) + citation count (0.10).

Credibility model (`credibility-model.ts`): Bayesian per-domain credibility (default 0.5). Confirmed claims → score toward 0.6; contradicted → toward 0.4. Backed by native Rust for fast updates.

Cap: 1000 findings maximum (trimmed by length). Results saved to `data/research/` as markdown.

Hypothesis engine (`hypothesis-engine.ts`): Generates competing hypotheses before searching. For a question like "is X causing Y?", the engine generates:

- **H1:** X causes Y directly
- **H2:** Y causes X (reverse causation)
- **H3:** Z causes both X and Y (confounder)
- **H4:** X and Y are uncorrelated (null hypothesis)

Each hypothesis gets Bayesian confidence updates based on found evidence, with `searchFor` and `searchAgainst` queries to avoid confirmation bias.

Fetch and extraction (`fetcher.ts`): Tiered fetch pipeline:

1. **Simple HTTP** for static pages (cheap, fast)
2. **Playwright rendering** for JS-heavy pages (slower, more expensive)
3. **Mozilla Readability** for article extraction
4. **unpdf** for PDF text extraction
5. **Cheerio** for DOM querying

Persistence (`persistence.ts`): Research runs are saved incrementally (every 10 findings) so a crash or session restart doesn't lose work. Each run gets a unique ID and is stored at `data/research/<id>/`:

- `query.txt` — original query
- `findings.json` — all findings with metadata
- `report.md` — final synthesized report
- `sources.json` — source credibility scores
- `timeline.json` — event log with timestamps

Research profiles (`profiles.ts`): Pre-configured search strategies for domains (`academic` , `news` , `technical` , `commercial` , `adversarial`). Each profile pre-weights search engines and sets domain filters.

Analysis engine (`analysis.ts`): Post-research analysis includes bias detection (one-sided evidence), gap identification (questions the research didn't answer), and follow-up suggestion.

3.12 Proactive Engine

Directory: `src/proactive/` (28+ files)

ATLAS can send messages without being prompted. The proactive system runs on three levels:

Heartbeat (`heartbeat.ts`): Fires on a configurable interval. Checks battery, CPU, network, disk. Sends morning briefing (random time 6–10 am). Triggers inner-thoughts, monitors, and dreaming checks.

Dreaming (`dreaming.ts`): When the user has been idle 2+ hours during 11 pm–6 am:

1. Memory consolidation (DreamReplay + SleepGate conflict detection + relationship decay)
2. Harvest pending tasks from conversation (LLM scans for unanswered questions, follow-ups)
3. Execute tasks via sub-agents (max 3 tasks, 10 min each, 30 min total budget)
4. Propose self-edits if ≥2 recent failures
5. Queue results for morning delivery

Observer pipeline (`observer.ts` , `composer.ts` , `dispatch.ts`): Intent sources → Observer queue → Composer (template fast-path or LLM fallback with Haiku) → Dispatch (kill-switch gates → channel sender).

Postures (per channel, configurable via `data/proactive-config.json`):

Posture	Max/day	Min gap	Heartbeat interval	Boundary only
calm	3	120 min	120 min	yes
active	8	45 min	60 min	no
ambient	1	360 min	240 min	yes

Defaults: Telegram = active, WhatsApp = calm, Overlay = ambient, Discord/Slack/Matrix/Email = calm.

Kill-switches: `ATLAS_PROACTIVE_DISABLE` (global off), `ATLAS_PROACTIVE_DISABLE_SOURCES` (CSV list of sources), `ATLAS_PROACTIVE_DISABLE_CHANNELS` (CSV list), `ATLAS_PROACTIVE_SHADOW` (compose + log but never send).

Rust scheduler (`proactive_scheduler.rs`): Intent queue per channel with priority formula: `base × freshness × posture_weight × source_weight` . Urgent=100, normal=10, ambient=1 base. Freshness decays exponentially.

Intent sources (`src/proactive/intent.ts`):

Source	What it watches
`latent_topic_monitor`	Topics the user has mentioned being interested in — signals when new relevant info appears
`memory_anticipation`	Memories with upcoming relevance (birthday tomorrow, expiring subscription)
`calendar_fact_check`	Upcoming calendar events — cross-references against stored facts to flag conflicts
`calendar_fusion`	Merges multiple calendar sources, detects double-booking
`stale_todo`	Todo items past their expected completion date
`meeting_followup`	After a meeting ends, suggests follow-up actions
`habit_anomaly`	Pattern break (e.g. user hasn't done their morning run for 3 days)
`env_context`	Environment signals (weather change, battery low, DND expiring)
`long_task_status`	Long-running tasks hitting milestones
`convex_notification`	Server-pushed notifications (credit low, account status change)

Composition fast-path — for template-friendly intents (low-novelty, high-frequency), the Composer uses a pre-written template with variable slots rather than calling the LLM. This cuts composition latency from ~800ms (LLM) to ~5ms (template) for the most common proactive messages. LLM fallback fires for high-novelty intents.

Post-filters: every composed message runs through banned-phrase and third-person filters before dispatch — prevents "As an AI assistant" style bot-speak and enforces first-person framing.

Mode emoji prefix — proactive messages prefix a mode indicator: 💡 for `fyi` , ❓ for `question` , ✅ for `review` . This lets users visually distinguish proactive from responsive messages.

3.13 Video Studio

Directory: `src/video-studio/` (15+ files), `src/tools/video-studio.ts`

A complete AI-guided video editing pipeline.

Workflow:

1. `video_create_project` — copies input files to `data/video-projects/vid_YYYY_MM_DD_NN/inputs/` ; SHA-256 integrity hashes stored
2. `video_analyze` — scene detection, keyframe captioning, Whisper transcription, face tracking, beat detection (Python sidecar)
3. `video_plan_edit` — LLM (Sonnet) generates a structured `EditPlan` in 2 passes with Zod validation
4. `video_render` — compiles the plan to FFmpeg filter graph; handles trim, scale, fade, overlays, transitions, audio mixing
5. `video_patch_plan` — RFC 6902 JSON Patch for incremental plan edits (replace/add/remove)
6. `video_ffmpeg` — raw FFmpeg escape hatch with security validation (no URL schemes, no path traversal, outputs restricted to `/tmp` or `data/video-projects/`)
7. `video_search_broll` — searches Internet Archive or Pexels (if `PEXELS_API_KEY` set)

EditPlan schema (`plan/schema.ts`): Includes output (aspect ratio 16:9/9:16/1:1/4:5, fps, resolution), sources with durations, video tracks (crop `auto_face` or `W:H:X:Y`), audio tracks (volume dB, beat sync, ducking), overlays (captions, text, stickers), transitions (xfade or cut).

Effects: `bg_replace:<path>` (downloads ~400 MB BiRefNet model), `color_grade:warm|cool|vintage|mono` , `stabilize` , `stylize:<preset>` .

Python sidecar: Beat detection, background matting, face tracking, and fast Whisper transcription run via Python scripts in `src/video-studio/python/` , called as child processes.

EditPlan validation pipeline (`plan/schema.ts`):

1. **Zod schema** — first validation pass; rejects plans with invalid types, missing required fields, out-of-range values
2. **Constraint checks** — referential integrity (all `source` IDs in tracks must exist in `sources[]`), non-overlapping video segments, audio volume sanity
3. **Duration check** — ensures `out > in` on every track, and that out values don't exceed the source's `duration`
4. **Overlay bounds** — overlay `from < to` , both within the output duration

A `validateEditPlan()` helper returns `{ ok: true, plan }` or `{ ok: false, errors: string[] }` with human-readable error messages.

Output targets:

Target	Format	Notes
`mp4`	H.264 + AAC	Direct render via FFmpeg
`fcpxml`	Final Cut Pro XML	For hand-tuning in NLE

Aspect ratios: `16:9` (YouTube/TV), `9:16` (TikTok/Reels/Shorts), `1:1` (Instagram feed), `4:5` (Instagram portrait).

Overlay types (discriminated union):

Type	Fields
`captions`	style, highlight_color, position, font_family/size
`text`	content, font color/family/size, box color, x/y
`sticker`	asset path, scale, x/y

Transition types: `xfade` (crossfade between clips, configurable duration), `cut` (hard cut).

3.14 Meta-Harness Optimizer

Directory: `src/harness/` (13 files)

An automated outer-loop optimization system (based on arxiv 2603.28052) that treats ATLAS configuration as a tunable parameter space and evolves it automatically.

30+ tunable dimensions (defined in `registry.ts`):

Dimension	Default	Range
`memory.fusion.relevanceWeight`	0.5	numeric
`memory.fusion.recencyWeight`	0.25	numeric
`memory.san.weight`	0.3	numeric
`memory.hopfield.weight`	0.15	numeric
`memory.recall.k`	10	3–50
`memory.crossEncoder.blend`	0.6	numeric

Dimension	Default	Range
`agent.contextCap`	8000	2000–20000
`evaluator.acceptThreshold`	70	50–95
`swarm.multiCriticThreshold`	60	30–90
`toolRouter.maxToolsTotal`	36	10–80
`research.discoveryPhaseRatio`	0.4	numeric
Sub-agent role system prompts	(defaults)	string
… and 20+ more		

Optimization loop (`optimizer.ts`):

1. **Phase 0:** Initialize 4 seed configs in parallel (baseline, memory-aggressive, tool-conservative, evaluator-strict)
2. **Phase 1 (per iteration):**
   - NSGA-II parent selection from Pareto frontier
   - Optionally generate TextGrad dimension critiques (if last candidate regressed ≥2 points)
   - Propose changes via **filesystem-agent** (code-proposer reads raw trace files — not summaries — giving 50% vs 34.9% accuracy per the paper ablation)
   - Validate code patches (static safety gates: no `eval` , no network, no destructive fs outside `data/harness/`)
   - GP surrogate pre-screening: skip evaluation if upper-confidence-bound < `best - 5`
   - Evaluate candidate (35% task + 25% memory + 25% tool + 15% token efficiency)
   - Update Pareto frontier; apply if improvement, revert if regression

Multi-objective tracking (`pareto.ts`): 4 objectives — task score (0–100), token efficiency (0–1), memory precision (0–1), tool accuracy (0–1). NSGA-II front peeling + crowding distance for selection diversity. Hypervolume approximation for frontier improvement detection.

Trace store (`trace-store.ts`): Every agent invocation is traced (tools selected, context sources, memory hits, evaluation score, duration). Traces are written per-candidate to `data/harness/runs/<run_id>/candidates/<id>/traces/*.json` for the filesystem-agent proposer to read directly.

Run storage layout:

```
data/harness/runs/<run_id>/
├── run_config.json
├── indexes/leaderboard.json
└── candidates/
    ├── c0000/            ← baseline
```

```
│   ├── manifest.json
│   ├── scores.json
│   ├── traces/
│   └── .metaharness/
└── c0001/...
```

Active config is persisted to `data/harness/active/harness-config.json` and loaded at boot.

Entry command: `/optimize-harness` slash command triggers the optimizer. Parameters:

- `budget` — maximum LLM spend in credits (default 10)
- `iterations` — max optimization iterations (default 20)
- `parallel` — parallel candidate evaluations (default 2)
- `seedDir` — initial candidate pool (default: 4 hand-crafted seeds)

TextGrad critique (`textgrad-refiner.ts`): When a candidate regresses, the optimizer asks the LLM to critique each dimension individually: "the `memory.recall.k` was set to 30 in this candidate — was that too aggressive? Too conservative? Just right?" The aggregated textual gradient is fed back to the proposer to guide the next proposal.

Filesystem-agent proposer — rather than feeding summary statistics to the LLM proposer, the optimizer gives it a filesystem of raw trace JSON files and lets it read them directly. The paper (arxiv 2603.28052) shows this improves proposal accuracy from 34.9% → 50%. The proposer sub-agent is restricted to read-only filesystem tools within the run directory.

Safety gates — before applying a candidate config, the optimizer runs static safety checks:

- No `eval` or `Function()` constructor
- No outbound network calls from modified code
- No filesystem writes outside `data/harness/`
- Type-checks pass (`tsc --noEmit`)
- Linter passes (`biome check`)

Candidates that fail safety gates are marked `scope-violation` and discarded without evaluation.

Leaderboard — `data/harness/runs/<run_id>/indexes/leaderboard.json` tracks the top-10 candidates by composite score, updated after every evaluation. The harness CLI can query the leaderboard to apply the current best config to production.

3.15 Voice Subsystem

Directory: `src/voice/` (9 files + `live/` subdirectory)

Talk mode (`talk-mode.ts`): Enables a continuous voice conversation. Flow: audio → transcribe → `runAgent()` → TTS → play.

Transcription (`transcribe.ts`): Tries in order — OpenAI Whisper or Groq `whisper-large-v3-turbo` → OpenRouter Gemini 2.5 Flash (with `input_audio` content). `transcribeAudio()` and `transcribeBuffer()` variants both supported.

TTS (`tts.ts`): OpenAI `tts-1` model. Default voice: `nova` . Other voices: `alloy` , `echo` , `fable` , `onyx` , `shimmer` . Output format: Opus.

Wake word (`wake-word.ts`): Trigger phrase detection on transcribed text. Current implementation uses substring matching; Picovoice Porcupine or Vosk integration is noted for future full VAD support.

Live voice mode (`live/`): Streaming bidirectional voice with tool calling.

Backend	Protocol	Notes
`gemini-live.ts`	`@google/genai` SDK, `Modality.AUDIO`	Model chain: gemini-3.1-flash-live-preview → gemini-2.5-flash-live-preview
`openai-realtime.ts`	WebSocket `wss://api.openai.com/v1/realtime`	Model: gpt-4o-realtime-preview
`hybrid-pipeline.ts`	STT → agent → TTS pipeline	Fallback when streaming backends unavailable

Voice-safe tools (allowed during live sessions): `spotify` , `music_control` , `set_volume` , `shell_exec` , `apple_script` , `web_search` , `get_current_time` , `memory_store` , `memory_search` , `schedule_task` , `set_reminder` , `read` , `write` , `send_file` .

Live voice config: `LIVE_VOICE_NAME` (Gemini voices: Aoede/Charon/Kore/Fenrir/Puck; OpenAI: alloy/shimmer/echo), `LIVE_VOICE_BACKEND` (gemini-live/openai-realtime/hybrid).

3.16 A2A Protocol

File: `src/agents/a2a.ts`

ATLAS implements Google's Agent-to-Agent (A2A) protocol, which enables external systems to discover and delegate tasks to ATLAS.

Agent Card (published at `/.well-known/agent.json`):

```
{
  "name": "A.T.L.A.S",
  "version": "0.2.0",
  "capabilities": ["research", "coding", "browser", "system", "memory"],
  "endpoint": "http://127.0.0.1:18792/a2a",
  "maxConcurrentTasks": 5
}
```

Task lifecycle: `pending` → `in_progress` → `completed` / `failed` / `cancelled`. Tasks auto-expire after 1 hour. Max 5 concurrent tasks.

Agent-as-tool pattern: Any A2A agent can be wrapped as a `delegate_to_{name}` tool and called from within ATLAS's own agent loop.

HTTP routes:

- `GET /.well-known/agent.json` — agent discovery
- `GET/POST /a2a/tasks` — list / create tasks
- `GET /a2a/tasks/:id` — task status
- `POST /a2a/tasks/:id/cancel` — cancel task

3.17 Communication Channels

Channel	Library	Notes
Telegram	grammY ^1.35	Sender ID allowlist; voice/image/video extraction; Markdown→HTML
Discord	discord.js ^14 + @discordjs/voice	Text + voice; message content intent
Slack	@slack/bolt ^4.7	Socket Mode
Matrix	matrix-js-sdk	Federated messaging
WhatsApp	@whiskeysockets/baileys ^7	4 identity tiers (owner/contact/unknown/group); persona system; rate limit 50/hr, min 2s interval
Email	IMAPFlow + Nodemailer	IMAP read + SMTP send; sender allowlist
Canvas	ws (WebSocket)	Port 18791 on `127.0.0.1`; CanvasWidget types: html/chart/table/form/markdown
Web Widget	Inline IIFE	Served at `/widget.js`; `?token=` auth; 56 px bubble, 350×500 window

---

Channel	Library	Notes
Desktop	Electron overlay	340 px panel at `http://127.0.0.1:18792` ; Chrome extension bridge at `ws://127.0.0.1:18792`

WhatsApp groups: Auto-approve if owner added; otherwise notifies owner with approve/decline prompts. Response gating: skip if < 30s since last send or bot share > 15% of total messages; then Haiku yes/no decision.

3.18 Desktop App (Electron)

Directory: `desktop/`

Cross-platform Electron app (macOS + Windows) with:

- **Overlay panel:** 340 px wide, floats on top of all windows, connects to the ATLAS server at `http://127.0.0.1:18792`
- **Auto-updater:** `electron-updater` ; provider = generic; update metadata served from Convex
- **Source protection:** Bytecode compilation via `bytenode` ; runtime XOR key fetched from Convex `/api/app/runtime-key` ; `.env.enc` encrypted with the runtime key
- **Session management:** Session token cached in Electron `safeStorage` with 1-hour refresh before expiry
- **Setup wizard:** `wizard.html` guides first-time config (Telegram token, user IDs, etc.)
- **Chrome extension bridge:** Companion extension (`chrome-extension/`) connects via WebSocket to `ws://127.0.0.1:18792` ; Manifest V3 service worker with 20s heartbeat

Build targets:

- macOS: DMG via `create-dmg.sh` (`npm run build:mac`)
- Windows: NSIS installer (`npm run build:win`)

3.19 Marketing Brain

Directory: `src/marketing/` (20 files)

A full content-marketing automation system for agencies and personal brands.

Platforms: Instagram, LinkedIn, TikTok, X (Twitter), Facebook, YouTube, Reddit, Product Hunt (9 platforms).

Core workflow:

1. `brand_audit` — scrapes website, analyzes social profiles, researches competitors
2. `campaign_plan` — generates content pillars, weekly cadence, platform calendar
3. Script writer — duration-aware scripts (Instagram 30s, TikTok 45s, YouTube Shorts 60s) with hook + scenes + hashtags
4. Video pipeline — uses Video Studio to generate clips; `video_plan_edit` → `video_render`
5. Social posting — `social_post` tool; platform adapters with rate limits (Instagram 25/24h, TikTok 15/24h, etc.)
6. Outreach engine — Apollo.io + Hunter.io for leads; email via Hostinger SMTP; Twitter DM (Pro tier)
7. Trend researcher — Apify `instagram-hashtag-scraper` + `clockworks/tiktok-scraper`

Content calendar: Stored in `marketing_calendar` SQLite table; statuses: draft → pending_approval → approved → scheduled → posted / failed.

Credential storage: Platform tokens stored as env vars (`IG_ACCESS_TOKEN_<brandId>`, `LI_ACCESS_TOKEN_<brandId>`, etc.) and managed via `credential-manager.ts`.

Lead finding (`outreach-engine.ts`, `lead-finder.ts`):

1. **Search Apollo.io** for people matching job-title + industry + location filters
2. **Verify emails via Hunter.io** — rejects role-based (`info@`, `support@`) and unverifiable addresses
3. **Enrich with LinkedIn data** (Phantombuster — optional, ToS risk)
4. **Segment into campaigns** — ICP scoring, persona tagging
5. **Draft personalized outreach** — LLM-written cold emails with context from the lead's profile
6. **Send via Hostinger SMTP** (bulk-safe, low deliverability risk) or platform DM

Rate limits per platform (conservative defaults):

Platform	Cap	Window
Instagram	25 posts	24 hours
TikTok	15 posts	24 hours
YouTube Shorts	10	24 hours
X (Twitter)	50 posts, 1000 DMs	24 hours
LinkedIn	20 posts	24 hours
Reddit	10 (tracks karma/age)	24 hours
Facebook	20 posts	24 hours
Product Hunt	1 launch	per product

Trend research (`trend-researcher.ts`): Uses Apify actors (`instagram-hashtag-scraper` , `clockworks/tiktok-scraper`) to pull top-performing content for given hashtags/keywords. Output is distilled into content ideas via the script writer LLM.

Brand audit (`brand-audit.ts`): Scrapes the user's website + social profiles + competitors, runs an LLM analysis for voice-of-brand, positioning gaps, and competitive differentiation. Output: a `BrandAudit` JSON blob with strengths/weaknesses/opportunities/threats.

Campaign planner (`campaign-planner.ts`): Given a brand audit and a marketing goal (awareness/leads/sales), generates a weekly calendar with:

- Content pillars (3–5 themes)
- Per-day platform mix (e.g. Mon=IG Reel + X, Tue=LinkedIn post, Wed=TikTok, etc.)
- Draft hooks and scripts for the first week

Product Hunt launch (`product-hunt.ts`): End-to-end launch automation — teaser posts, hunter outreach, launch-day crew coordination, comment monitoring, post-launch analytics.

---

4. Data Flow

Inbound message (Telegram example)

```
User sends message on Telegram
    |
    ▼
telegram.ts: sender ID allowlist check → extract media (voice/image/video)
    |
    ▼
runAgent(llm, message, onProgress, media)
    |
    ├─ [First iteration] 12 memory systems fire in parallel
    |       identity + traits + session + emotional + episodic +
relationship
    |       + anticipation + core memory + HNSW recall + task learnings +
KG + self-state
    |       → inject ~2-3K tokens of context
    |
    ├─ chooseModel() → Rust router → model + thinking level
    |
    ├─ tool-router: embed query → cosine similarity → load top-K tool
schemas
```

```
    │
    ├─ encryptedFetch() → POST /api/llm/chat on Convex
    │       → Convex: decrypt request → assemble system prompt → forward
to OpenRouter
    │       → OpenRouter → LLM provider → response
    │       → Convex: encrypt response → return
    │
    ├─ parse tool calls → execute in parallel (except browser: sequential)
    │       each tool result → feed back into messages array
    │
    ├─ check finish_reason / TASK_COMPLETE → loop or exit
    │
    └─ [After session] runReflection() → SAC-DR → embed conversation →
store
    │
    ▼
smartChunkMessage() → Markdown→HTML → send via Telegram API
```

Credit flow

```
User message → account guard checks local cache
    → if stale: GET /api/account/status on Convex
    → Convex: validateSession → return {credits, status}
    → block if credits < minimum threshold
    → on LLM call: Convex deductCredits() atomically (OCC-serialized)
    → local balance cache updated
```

---

5. Security Architecture

Security is defense in depth. Even if any single layer fails, the next layer contains the blast radius.

5.1 End-to-End Encryption (Application Layer)

Every LLM call is encrypted at the application layer on top of HTTPS:

Step 1 — Session setup (ECDH P-256):

```
Client:  _ecdh.generateKeys()  → pubKey_C  (base64)
         POST /api/auth/session { pubKey: pubKey_C }
Server:  _ecdh.generateKeys()  → pubKey_S
         shared = _ecdh.computeSecret(pubKey_C)
```

```
          returns { pubKey: pubKey_S, token, sharedKey_id }

Both:     salt = b"atlas-e2e-salt"
          info = b"atlas-e2e-aes256-key"
          key  = SHA-256(shared || salt || info)   ← 256-bit AES key
```

Step 2 — Per-request envelope (AES-256-GCM):

```
envelope = {
  iv:  random 12 bytes (base64)
  ct:  AES-256-GCM ciphertext (base64)
  tag: 16-byte auth tag (base64)
  ts:  Unix ms timestamp
  rid: random request id (prevents replay within 90s window)
}
```

Step 3 — Replay protection: server rejects any request whose `ts` is outside the ±90-second window. The `rid` is tracked server-side to catch duplicates within that window.

Step 4 — Response encryption: the server encrypts the response with the same derived key before returning. The client verifies the auth tag and decrypts before parsing.

API keys (OpenRouter, Google, Deepgram, Apify, Apollo, Hunter, CAPTCHA services, etc.) live only in Convex environment variables. They are never sent to the client in any form.

5.2 Docker Sandbox

Untrusted code execution (shell tools with unknown payloads, tool fabricator sandbox tests, `code_execute` from the code-action agent) runs in a tightly constrained Docker container:

Setting	Value
Base image	`node:20-alpine`
Memory limit	256 MB
CPU quota	50% (`CpuPeriod=100000` , `CpuQuota=50000`)
Network mode	`none` (no outbound connections at all)
AutoRemove	`true`
Default timeout	30 seconds
Read-only root filesystem	yes
Dropped capabilities	ALL

Setting	Value
Work directory	`/sandbox` mounted tmpfs

The sandbox is disabled entirely if the Docker daemon is not available — `src/security/sandbox.ts` falls back to refusing untrusted execution rather than running it unsandboxed.

5.3 Protected Paths & Destructive Command Allowlist

`src/security/allowlist.ts` blocks any shell command whose pattern matches:

- Recursive root delete (`rm -rf /`, `rm -rf /*`, `rm -rf ~`)
- Disk wipe utilities (`diskutil eraseDisk`, `format C:`, `mkfs.*`)
- Raw block device writes (`dd of=/dev/disk0`, `dd of=/dev/sd*`)
- Fork bombs (`:(){ :|:& };:`, `perl -e 'fork while 1'`)
- Power/reset (`halt`, `shutdown`, `reboot`, `init 0`)
- Destructive writes to ATLAS's own installation directory

ATLAS's own data directory (`data/`, `dist/`, `src/`, `native/`) is also protected from self-modification unless the user explicitly opts in.

5.4 Secrets Storage

`src/security/secrets.ts` encrypts any user-provided secret before writing to disk:

- Algorithm: AES-256-GCM
- KDF: `scrypt` with salt "atlas-salt", 32-byte key
- Format: `iv:tag:ciphertext` (all hex)
- Location: `data/secrets.enc`

The key is derived at runtime from a machine-stable seed — compromising the encrypted file alone is not enough to read secrets; the attacker also needs the host machine.

5.5 Rate Limiting

Convex enforces a sliding-window rate limit on the `/api/llm/chat` endpoint: **60 requests per 60-second window per session token**. Violations return HTTP 429 with a `Retry-After` header.

Client-side, individual tools also self-rate-limit to prevent single-tool storms (e.g. `web_search` is capped at 30 calls per minute).

5.6 Session Security

- **Tokens:** 256-bit cryptographically random hex strings (generated via `crypto.randomBytes(32).toString('hex')`)
- **TTL:** 24 hours, refreshable by calling `/api/auth/refresh` up to 1 hour before expiry
- **Device fingerprint binding:** optional; when enabled, a session token can only be used from the device that created it
- **Revocation:** all sessions for a user can be revoked with a single call to `/api/auth/revoke?all=true`
- **Timing-safe comparison:** `timingSafeCompare()` is used for key validation to prevent timing attacks
- **Secure storage:** on desktop, session tokens are cached in Electron `safeStorage` (Keychain on macOS, DPAPI on Windows)

5.7 Other Security Measures

- **Dangerous tool gating:** tools with `requiresConfirmation: true` (delete, format, send_email, git push —force) are held by `human-interrupt.ts` for user approval; default 5-minute timeout with configurable default action
- **Write-path gating in plan mode:** `edit` and `write` tool paths are runtime-validated against the active plan-mode state (`plan`, `spec`, `awaiting_approval`) — not just prompt-enforced
- **MCP tool vetting:** external MCP server tool schemas are validated against a JSON Schema before being exposed to the LLM
- **Audit log:** every tool call is appended to `data/audit.jsonl` with timestamp, tool name, args (redacted), and outcome; HMAC-signed on reflection events for tamper detection

---

6. Deployment

Local-first design

ATLAS is designed to run on the user's own machine. All memory (SQLite, embeddings, browser profile, research results) stays local. The Convex backend is the only external dependency.

Convex environments

Environment	URL	When used
Development	`zany-kudu-607.convex.cloud`	Default (`ATLAS_ENV` not set)

Environment	URL	When used
Production	`avid-oyster-465.convex.cloud`	`ATLAS_ENV=prod`
Custom	`CONVEX_URL` / `CONVEX_SITE_URL`	Override either individually

Platform binaries (native Rust)

The native module is pre-compiled per platform. Falls back to pure-JS if unavailable.

Platform	Binary filename
macOS Apple Silicon	`atlas-native.darwin-arm64.node`
macOS Intel	`atlas-native.darwin-x64.node`
Windows x64	`atlas-native.win32-x64-msvc.node`
Linux x64	`atlas-native.linux-x64-gnu.node`
Linux ARM64	`atlas-native.aarch64-unknown-linux-gnu.node`

Build commands

```
# Install Node.js dependencies
npm install

# Build Rust native module (requires Rust toolchain)
npm run build:native

# Compile Swift helpers (macOS only)
npm run build:ocr         # OCR screen helper
npm run build:location    # GPS/location helper

# TypeScript compilation
npm run build             # tsc → dist/

# Everything at once
npm run build:all

# Start Convex dev backend
npx convex dev

# Run in development (hot reload)
npm run dev
```

```
# Run in production
npm start
```

Desktop packaging

```
cd desktop && npm install

npm run build:mac    # macOS DMG (uses create-dmg.sh)
npm run build:win    # Windows NSIS installer
```

System requirements

Component	Minimum
Node.js	>= 20
Rust toolchain	Latest stable (for native build)
RAM	2 GB recommended
macOS	13+ (for Swift helpers, overlay app)
Docker	Optional (for sandboxed code execution)

---

7. Configuration Reference

All configuration is via `.env` (copy from `.env.example`).

Required

Variable	Description
`TELEGRAM_BOT_TOKEN`	Bot token from @BotFather
`ALLOWED_USER_IDS`	Comma-separated Telegram user IDs allowed to use the bot
`ATLAS_OTPLESS_USER_ID`	OTPless user ID for account authentication

Optional — Channels

Variable	Description
`EMAIL_USER`	IMAP/SMTP email address
`EMAIL_PASSWORD`	Email app password
`EMAIL_WHITELIST`	Comma-separated allowed sender addresses
`DISCORD_BOT_TOKEN`	Discord bot token
`DISCORD_CLIENT_ID`	Discord application ID
`DISCORD_ALLOWED_USER_IDS`	Allowed Discord user IDs
`DISCORD_NOTIFICATIONS_CHANNEL_ID`	Channel for proactive messages
`SLACK_BOT_TOKEN`	Slack bot token
`SLACK_APP_TOKEN`	Slack Socket Mode app token
`SLACK_ALLOWED_USER_IDS`	Allowed Slack user IDs
`SLACK_DEFAULT_CHANNEL`	Default Slack channel
`MATRIX_HOMESERVER`	Matrix homeserver URL
`MATRIX_USER_ID`	Matrix user ID
`MATRIX_PASSWORD`	Matrix password
`MATRIX_ALLOWED_ROOMS`	Allowed room IDs
`PRIMARY_CHANNEL`	Default channel for proactive messages (default: `telegram`)
`WIDGET_TOKEN`	Authentication token for web widget

Optional — AI / Models

Variable	Description
`ATLAS_EMBEDDING_MODEL`	`minilm` (default, 384-dim) or `nomic` (768→256-dim)
`HF_TOKEN`	HuggingFace token (required for Nomic gated model)
`LIVE_VOICE_NAME`	Voice name for live voice mode
`LIVE_VOICE_BACKEND`	`gemini-live`, `openai-realtime`, or `hybrid`
`NOPECHA_API_KEY`	Free CAPTCHA solving (100/day)
`CAPTCHA_API_KEY`	2captcha paid API key
`PEXELS_API_KEY`	Pexels stock footage for Video Studio

Variable	Description
RUNWAY_API_KEY	Runway ML for AI video generation

Optional — Proactive kill-switches

Variable	Effect
ATLAS_PROACTIVE_DISABLE=true	Globally disable all proactive messages
ATLAS_PROACTIVE_DISABLE_SOURCES=csv	Disable specific intent sources
ATLAS_PROACTIVE_DISABLE_CHANNELS=csv	Disable specific channels
ATLAS_PROACTIVE_SHADOW=true	Compose messages but never send (debug mode)
ATLAS_PROACTIVE_DEBUG=false	Suppress proactive debug logs

Optional — Feature flags

Variable	Effect
ATLAS_ENV=prod	Use production Convex deployment
ATLAS_AGENT_WORKTREE=true	Enable git worktree isolation for parallel coders
ATLAS_NO_CODE_CRITIC=true	Skip compile/test/lint/SAST in swarm synthesis
ATLAS_NO_CODEBASE_INDEX=true	Disable codebase indexing
ATLAS_NO_POST_EDIT_DIAGNOSTICS=true	Don't attach LSP diagnostics to edit results
ATLAS_NO_CODING_THINKING=true	Disable extended thinking for coding tasks
ATLAS_NO_CODING_ESCALATION=true	Don't force Pro tier for coding
ATLAS_INDEX_DEPTH=symbols|imports|calls	Override auto-depth picker
ATLAS_CODE_PARSER=regex	Force legacy regex extractors (skip tree-sitter)
ATLAS_MULTI_TREE_SITTER=true	Enable Python/Rust/Go tree-sitter (unstable)
ATLAS_TOOL_DEBUG=off|verbose	Tool breadcrumb verbosity
ATLAS_POST_SYNTHESIS_BUDGET_MS	Wall-clock cap for code-critic + multi-critic (default 90000)

Optional — Voice

Variable	Description
`LIVE_VOICE_NAME`	Gemini voices: Aoede, Charon, Kore, Fenrir, Puck. OpenAI voices: alloy, shimmer, echo
`LIVE_VOICE_BACKEND`	Force backend: `gemini-live`, `openai-realtime`, or `hybrid` (auto-detect if empty)
`LIVE_VOICE_ENABLED`	Set to `false` to disable live voice entirely
`WAKE_WORD_ENABLED`	Enable wake word detection (default `false`)

Note: `GOOGLE_API_KEY` and `OPENAI_API_KEY` for live voice backends are stored in Convex env vars (server-side) and fetched via `/api/keys/voice`.

Optional — Marketing Brain

Replace `{brand}` with your brand slug (lowercase, hyphens). Example: `IG_USER_ID_acme-io=123456789`.

Instagram (Meta Graph API):

Variable	Description
`IG_USER_ID_{brand}`	Instagram Business Account ID
`IG_ACCESS_TOKEN_{brand}`	Meta Page Access Token (60-day, auto-refresh)

LinkedIn:

Variable	Description
`LI_ACCESS_TOKEN_{brand}`	LinkedIn API access token
`LI_ORG_ID_{brand}`	Company Page numeric ID (optional — for org posts)

TikTok, X (Twitter), Facebook, YouTube:

Variable	Description
`TIKTOK_ACCESS_TOKEN_{brand}`	TikTok API access token
`X_API_KEY_{brand}`	Twitter/X API key
`X_API_SECRET_{brand}`	Twitter/X API secret
`X_ACCESS_TOKEN_{brand}`	OAuth access token

Variable	Description
`X_ACCESS_SECRET_{brand}`	OAuth access token secret
`FB_PAGE_ID_{brand}`	Facebook Page ID
`FB_ACCESS_TOKEN_{brand}`	Facebook Page access token

Reddit:

Variable	Description
`REDDIT_CLIENT_ID_{brand}`	Reddit app client ID
`REDDIT_CLIENT_SECRET_{brand}`	Reddit app client secret
`REDDIT_REFRESH_TOKEN_{brand}`	Long-lived refresh token

Product Hunt & shared services:

Variable	Description
`PH_API_TOKEN`	Product Hunt API token
`APIFY_TOKEN`	Apify — trend research + competitor scraping
`APOLLO_API_KEY`	Apollo.io — lead finding
`HUNTER_API_KEY`	Hunter.io — email verification
`RUNWAY_API_KEY`	Runway ML — AI video generation
`PHANTOMBUSTER_API_KEY`	LinkedIn automation (optional, ToS risk)
`PEXELS_API_KEY`	Stock video fallback (optional — raises rate limits)

Optional — Developer / Debugging

These flags are intended for developers working on ATLAS itself. All default OFF (feature enabled).

Variable	Effect
`ATLAS_DATA_DIR`	Override the `data/` directory location
`ATLAS_USER_DATA_DIR`	Electron `userData` path (used for wizard seed)
`ATLAS_WORKSPACE`	Override workspace root (default: `~/Desktop/A.T.L.A.S Files`)
`MEMORY_DEBUG=true`	Verbose memory subsystem logging

Variable	Effect
`ATLAS_TOOL_DEBUG=off|verbose`	Tool breadcrumb verbosity
`ATLAS_TS_DEBUG=true`	Verbose tree-sitter grammar-load logging
`ATLAS_DISABLE_BUILTIN_HOOKS=true`	Drop the built-in LSP pre-edit PreToolUse hook
`ATLAS_NO_ARCHITECTURE_INJECTION=true`	Skip `<architecture>` block on first iteration
`ATLAS_NO_REPO_MAP=true`	Skip `<repo_map>` block on first iteration
`ATLAS_NO_SAST=true`	Skip Semgrep in code-critic
`ATLAS_NO_AGENT_WORKTREE=true`	Force-off git worktree isolation even if `ATLAS_AGENT_WORKTREE=true`
`ATLAS_WAVE_SPECULATION_CAP`	Integer (default `1`) or `off` — cap on concurrent speculative executions per wave
`ATLAS_NO_CODER_SPECULATION=true`	Disable racing alternative approaches on coder steps
`ATLAS_POST_SYNTHESIS_BUDGET_MS`	Wall-clock cap on code-critic + multi-critic + MoA chain (default 90000 ms)
`ATLAS_PROACTIVE_V2_SCHEDULER=false`	Fall back to Composer-only dispatch (no scheduler, no inbox)
`ATLAS_PROACTIVE_V2_INBOX=true`	Enable the Phase 4 proactive inbox
`ATLAS_AUTO_PLAN_MODE=true`	Opt-in automatic entry into plan mode on first user message
`ATLAS_INDEX_ASYNC=force|never`	Override the block-vs-async codebase index timing picker
`CONVEX_URL`	Override Convex deployment URL (both dev and prod)
`CONVEX_SITE_URL`	Override Convex HTTP site URL

Default-on kill-switches summary

The environment variables listed below all default to the feature being ON. Setting any of them to `"true"` disables the feature. Use this to regression-isolate when a subsystem misbehaves:

- Coding escalation: `ATLAS_NO_CODING_ESCALATION`
- Extended thinking for coding: `ATLAS_NO_CODING_THINKING`
- Coder speculation: `ATLAS_NO_CODER_SPECULATION`

- Git worktree isolation: `ATLAS_NO_AGENT_WORKTREE` (note the inverse — the positive flag is `ATLAS_AGENT_WORKTREE=true`)
- Built-in PreToolUse hooks: `ATLAS_DISABLE_BUILTIN_HOOKS`
- Code critic: `ATLAS_NO_CODE_CRITIC`
- Post-edit LSP diagnostics: `ATLAS_NO_POST_EDIT_DIAGNOSTICS`
- SAST (Semgrep): `ATLAS_NO_SAST`
- Codebase index: `ATLAS_NO_CODEBASE_INDEX`
- Architecture injection: `ATLAS_NO_ARCHITECTURE_INJECTION`
- Repo map injection: `ATLAS_NO_REPO_MAP`
- Proactive globally: `ATLAS_PROACTIVE_DISABLE`

Appendix A — File Tree Summary

```
ATLAS/
├── src/                          (TypeScript source, ~385 files)
│   ├── index.ts                  (entry point, boot sequence)
│   ├── agent.ts                  (runAgent, core loop)
│   ├── llm.ts                    (LLM client interface)
│   ├── crypto-transport.ts       (ECDH + AES-256-GCM)
│   ├── model-router.ts           (Rust wrapper)
│   ├── tool-router.ts            (semantic tool selection)
│   ├── memory/                   (39 files)
│   ├── agents/                   (30 files — swarm, mesh, long-task, sub-agent, etc.)
│   ├── tools/                    (~100+ files)
│   │   ├── browser/              (20 files — dual-backend automation)
│   │   └── coding/               (25+ files — design docs, LSP, codebase index)
│   ├── research/                 (11 files — deep research engine)
│   ├── proactive/                (28+ files — heartbeat, dreaming, composer)
│   ├── harness/                  (13 files — Meta-Harness optimizer)
│   ├── video-studio/             (15+ files — AI video editing)
│   ├── marketing/                (20 files — Marketing Brain)
│   ├── voice/                    (9 files + live/ — TTS, STT, live voice)
│   ├── social/                   (4 files — theory of mind, emotion, pragmatics)
│   ├── planning/                 (MCTS, hierarchical)
│   ├── mcp/                      (Model Context Protocol)
```

```
│   └── security/                (sandbox, allowlist, secrets)
├── native/                      (Rust, 55+ modules)
│   ├── src/
│   └── Cargo.toml
├── convex/                      (serverless backend)
│   ├── schema.ts
│   ├── http.ts
│   ├── crypto.ts
│   └── store/
├── desktop/                     (Electron app)
├── chrome-extension/            (MV3 companion extension)
├── overlay-app/                 (native macOS Swift app)
├── portals/
│   ├── admin-panel/             (Next.js admin dashboard)
│   └── developer-portal/        (Next.js developer portal)
├── skills/                      (built-in markdown skills)
├── prompts/                     (server-side prompt source)
├── scripts/                     (Swift helpers, seed scripts)
├── data/                        (runtime state — gitignored)
│   ├── memories.db              (SQLite, WAL)
│   ├── codebase-index/
│   ├── agent-worktrees/
│   ├── long-tasks/
│   ├── research/
│   ├── browser-profile/
│   ├── video-projects/
│   ├── harness/runs/
│   ├── proactive-config.json
│   └── secrets.enc
└── .env                         (user config)
```

Appendix B — Performance Highlights

- **Vector ops (native Rust):** 15–20× faster than pure JS for cosine similarity, top-K search, batch similarity
- **Tool routing:** semantic selection via MiniLM embeddings loads only the needed ~15–36 tools per request (out of 100+)
- **Memory fan-out:** 12 retrieval systems execute in parallel, total latency bounded by the slowest single retrieval (~150–300 ms in practice)

- **Prompt caching:** Anthropic cache breakpoints on system prompt + tools schema + conversation history reduce token costs by up to 90% on repeated context
- **LLM response cache:** semantic LRU with cosine > 0.88 and 3-minute TTL catches exact repeats and near-paraphrases of simple queries
- **Incremental codebase index:** Merkle-hashed file tracking means a refresh after small edits touches only changed files
- **Model cascading:** trivial tasks hit Haiku (0.25 credits); complex tasks auto-escalate to Opus (3.0 credits) — average spend stays low while peak capability is always reachable
- **Tool result truncation:** per-tool smart truncation (shell keeps tail, JSON prunes trees, browser keeps IDs, files keep head + tail) prevents token-bloat feedback loops

---

Appendix C — Glossary

- **ATLAS** — Advanced Technology for Logical Autonomous Systems
- **ECDH** — Elliptic-Curve Diffie-Hellman key exchange
- **AES-256-GCM** — Authenticated encryption with associated data, 256-bit key, Galois/Counter Mode
- **HNSW** — Hierarchical Navigable Small World (approximate nearest-neighbor search)
- **FTS5** — SQLite Full-Text Search version 5 (with BM25 ranking)
- **BM25** — Best Match 25, a probabilistic relevance ranking function
- **MiniLM** — small sentence-transformer model (all-MiniLM-L6-v2, 384-dim embeddings)
- **SAST** — Static Application Security Testing (Semgrep in ATLAS)
- **LSP** — Language Server Protocol
- **MCP** — Model Context Protocol
- **OCC** — Optimistic Concurrency Control (used by Convex for credit deduction)
- **RRF** — Reciprocal Rank Fusion (blends multiple ranked lists)
- **OTPless** — third-party authentication service used for account identity
- **NSGA-II** — Non-dominated Sorting Genetic Algorithm II (multi-objective optimization)
- **TextGrad** — Text-based gradient analogue for LLM output critique
- **SAC-DR** — Self-Alignment via Cognitive Dissonance Resolution (reflection algorithm)
- **SOFAI-LM** — Slow-Fast AI for Language Models (metacognition framework)
- **SAN** — Spreading Activation Network (memory retrieval boost)
- **A2A** — Agent-to-Agent protocol (Google)
- **DAG** — Directed Acyclic Graph
- **WAL** — Write-Ahead Logging (SQLite mode)
- **DND** — Do Not Disturb (proactive silencing)
- **XOR key** — runtime key used to decrypt `.env.enc` in the desktop app

---

Copyright © 2024-2026 XAGI Labs Private Limited. All rights reserved.

Website: tryatlasagi.com · xagilab.com